

Feistel Network

This worksheet explains why DES is designed as a Feistel network and what problems that design solves.

We will

- Describe what a Feistel network is
- Explain how encryption and decryption work in a Feistel cipher
- Understand why Feistel structure was chosen for DES
- Connect Feistel rounds to confusion and diffusion
- Explain why Feistel networks make decryption easy

1. The Problem → DES Needs to Solve

DES is a block cipher, so it must:

- Encrypt fixed-size blocks (64 bits)
- Use the same secret key for encryption and decryption
- Be efficient in hardware
- Be reversible (decryption must undo encryption)

A naive approach would be:

- Scramble all 64 bits at once
- Then try to reverse the process

But designing a complex function that is both secure and easily reversible is very hard.

DES solves this problem by using a Feistel network.

2. What Is a Feistel Network?

A Feistel network is a general **design pattern for** building a reversible block cipher.

In simple terms:

A Feistel network is a way to encrypt data so that it can always be decrypted, even if the internal operations themselves are not reversible.

This is the key idea that makes Feistel networks powerful.

The basic structure of a Feistel network is:

1. Split the block into two halves
2. Apply a function to one half
3. Mix the result into the other half using XOR

4. Swap the halves
5. Repeat for many rounds

Because of how XOR and swapping work, this structure guarantees that encryption can always be undone.

3. Splitting the Block

DES starts by splitting its 64-bit block into two halves:

Left (L0) = first 32 bits

Right (R0) = last 32 bits

Only half of the data is processed by the round function at a time.

This immediately simplifies design and improves efficiency.

4. The Feistel Round Structure

A single Feistel round looks like this:

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i \oplus F(R_i, K_i)$$

Where:

- L_i and R_i are the left and right halves entering round i
- K_i is the round key
- F is the round function

Important observation:

The function F does not need to be reversible.

A function is reversible if you can undo it and recover the original input using the same rules.

Examples:

- XOR is reversible: $A \oplus B \oplus B = A$
- Addition modulo 256 is reversible if you know what was added

Examples of non-reversible operations:

- Many substitutions
- Hash functions
- One-way scrambling steps

In a Feistel network, even if the function F itself cannot be reversed, the *overall cipher* can still be decrypted. This is a major advantage of the Feistel design.

5. What the Round Function Does in DES

In DES, the function F:

- Expands the right half
- XORs it with the round key
- Applies S-box substitutions (confusion)
- Applies a permutation (diffusion)

So all the cryptographic complexity lives inside F.

6. Where Confusion and Diffusion Appear

In each Feistel round:

- **Confusion** comes from S-box substitutions inside F
- **Diffusion** comes from permutations inside F and from swapping halves across rounds

Over many rounds:

- Each plaintext bit influences many ciphertext bits
- Each key bit influences many ciphertext bits

This is how avalanche emerges.

7. Why Swapping Halves Matters

At the end of each round:

- The right half becomes the new left half
- The modified left half becomes the new right half

This swap:

- Forces information to move across the block
- Ensures both halves are fully affected over time

Without the swap, diffusion would be much weaker.

8. Why Feistel Networks Make Decryption Easy

A key advantage of Feistel networks is that decryption uses the same structure as encryption.

The only difference:

- Round keys are applied in reverse order

Because XOR is reversible:

$$A \oplus B \oplus B = A$$

The Feistel structure guarantees that each round can be undone.

9. Why DES is a Feistel Network

DES was designed in the 1970s, with strong constraints:

- Efficient hardware implementation
- Simple control logic
- Limited chip area
- Predictable timing

Feistel networks offered:

- Reversibility without complex inverse functions
- Reuse of the same round hardware
- A clean separation between structure and cryptographic strength

This made DES practical and secure for its time.

DES is a Feistel network because:

- It allows strong confusion and diffusion
- It guarantees reversibility
- It simplifies decryption
- It scales well with multiple rounds

Modern block ciphers may use different structures, but the Feistel idea remains foundational.

Key Expansion and Round Key Generation (Toy Example)

Step 1: Start With a 12-Bit Encryption Key

Assume the main encryption key is:

$$K = 1101\ 0110\ 1011$$

This is the *only* secret value.

Step 2: Expand the Key to 16 Bits

Because our block size is 16 bits, we first expand the key using a **simple toy rule**:

Repeat the first 4 bits of the key at the end.

Original key: 1101 0110 1011

Expanded key: 1101 0110 1011 1101

This expansion step:

- Aligns the key size with the block size
- Does not add security by itself
- Exists only for demonstration purposes

Step 3: Generate a Round Key

To create a round key, we apply a small transformation to the expanded key.

Toy rule:

1. Split the expanded key into two halves
- 2. Rotate each half left by 1 bit**
3. Take the first 8 bits as the round key

Apply the rule:

Split:

Left = 1101 0110

Right = 1011 1101

Rotate left by 1 bit:

Left = 1011 0110

Right = 0111 1011

Select round key (first 8 bits):

Round Key K1 = 1011 0110

This round key will be used in the Feistel round.

Numeric Example:

This example shows, step by step:

1. How a 12-bit main key can be expanded to match a 16-bit block size (toy rule)
2. How a round key can be generated from that expanded key (toy key schedule)

3. How decryption works without inverting the round function

This is a teaching example. Real DES uses a different key schedule and different sizes.

Given Values

Plaintext block (16 bits):

P = 1010 1100 0011 0110

Split into halves (8 bits each):

L0 = 1010 1100

R0 = 0011 0110

Main encryption key (12 bits):

K_main = 1101 0110 1011

Step A: Expand the 12-bit Main Key to 16 Bits (Toy Rule)

Toy expansion rule:

Repeat the first 4 bits of the 12-bit key.

K_main = 1101 0110 1011

K_exp16 = 1101 0110 1011 1101

This expansion is only to make the example easy to follow.

Step B: Generate a Round Key From the Expanded Key (Toy Key Schedule)

Toy key schedule rule (simple and visible):

1. Split the 16-bit expanded key into two 8-bit halves
2. Rotate each half left by 1 bit
3. Use the rotated **right half** as the **round key** (8 bits)

1) Split

K_exp16 = 1101 0110 | 1011 1101

Left(KL) | Right(KR)

KL = 1101 0110

KR = 1011 1101

2) Rotate Left by 1

rot1(KL) = 1010 1101

rot1(KR) = 0111 1011

3) Choose Round Key (8 bits)

Toy choice:

Round Key $K1 = \text{rot1}(KR)$

$K1 = 0111\ 1011$

Now we have a round key to use inside the Feistel round.

Step C: Encryption (One Feistel Round)

We will use a simple round function:

$$F(R, K1) = R \oplus K1$$

1) Compute $F(R0, K1)$

$R0 = 0011\ 0110$

$K1 = 0111\ 1011$

$$F = 0100\ 1101$$

2) Compute New Halves (Feistel Rules)

Feistel update:

$L1 = R0$

$R1 = L0 \oplus F(R0, K1)$

Apply:

$L1 = 0011\ 0110$

$L0 = 1010\ 1100 \oplus F = 0100\ 1101 \rightarrow R1 = 1110\ 0001$

Ciphertext after one round:

$C = L1 \mid R1 = 0011\ 0110\ 1110\ 0001$

Step D: Decryption (Recovering the Plaintext Without Inverting F)

Start with ciphertext halves:

$L1 = 0011\ 0110$

$R1 = 1110\ 0001$

1) Recover $R0$

From Feistel structure:

$R0 = L1 = 0011\ 0110$

2) Recompute $F(R0, K1)$

(We do not invert F; we just compute it again.)

$R0 = 0011\ 0110 \oplus K1 = 0111\ 1011 \rightarrow F = 0100\ 1101$

3) Recover L0

From encryption:

$$R1 = L0 \oplus F(R0, K1)$$

Undo with XOR:

$$L0 = R1 \oplus F(R0, K1)$$

Apply:

$$R1 = 1110\ 0001 \oplus F = 0100\ 1101 \rightarrow L0 = 1010\ 1100$$

Recovered plaintext halves:

$$L0 = 1010\ 1100$$

$$R0 = 0011\ 0110$$

Recombined plaintext:

$$P = 1010\ 1100\ 0011\ 0110$$

Original plaintext recovered without reversing F.

Note that

- The round function F was never inverted
- We only recomputed F and XORed it away
- XOR and swapping preserve all information

This is why Feistel networks guarantee reversibility.

A Feistel cipher is reversible because it undoes the XOR mixing, not the round function itself.